Answering Federated Range Queries with Local Differential Privacy

Yuemin Zhang
Hong Kong Polytechnic University
ymin.zhang@connect.polyu.hk

Qingqing Ye†
Hong Kong Polytechnic University
qqing.ye@polyu.edu.hk

Junxu Liu
Hong Kong Polytechnic University
junxu.liu@polyu.edu.hk

Wei Dong
Nanyang Technological University
wei_dong@ntu.edu.sg

Haibo Hu
Hong Kong Polytechnic University
haibo.hu@polyu.edu.hk

ABSTRACT

Answering range queries under local differential privacy (LDP) is a well-studied problem, which enables service providers to gain insights into decentralized data distributions. However, existing approaches operate exclusively in a single-party setting and prove challenging to extend to practical cross-party environments. As such, we study the problem of *federated range queries* under LDP, which addresses cross-party range queries in decentralized settings. Applying LDP to federated range queries amplifies statistical heterogeneity challenges, as utility deterioration stems from the accumulated LDP noise, and the unrealistic distribution (e.g., uniform or linear) assumption in existing methods. Our solution, the private cubic spline (PrivCS) mechanism, employs a hierarchical interval tree structure to model the underlying data distribution and accurately answer arbitrary range queries, while satisfying ϵ-LDP. Additionally, we introduce the collaborative frequency filtering aggregation that adaptively calibrates and masks frequency estimates by leveraging relationships among across-party frequencies and analyzing pruning bias, thereby reducing the cumulative noise. To the best of our knowledge, this is the first study to explore federated range queries in cross-party scenarios, under the stringent LDP guarantee. Comprehensive experiments on both real-world and synthetic datasets demonstrate the effectiveness of our mechanism.

PVLDB Reference Format:
Yuemin Zhang, Qingqing Ye, Junxu Liu, Wei Dong, and Haibo Hu. Answering Federated Range Queries with Local Differential Privacy. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

As a fundamental and prevalent operation for computing essential statistics of user populations, the range query calculates the proportion of users whose input falls within a specified interval. Nevertheless, since individual data often contains sensitive personal information, users are typically unwilling to share their raw data.

Local differential privacy (LDP) [12], the *de facto* privacy standard for decentralized settings, enables users to perturb their data locally before transmitting sanitized versions to data collectors. This makes LDP particularly well-suited for privacy-preserving data analytics. Numerous studies have explored LDP-based range queries [3, 7, 26, 27, 29, 31, 34]. However, these approaches operate exclusively in a single-party setting, where all users belong to the same party and the data collector receives sanitized data directly from them.

Practical deployments, conversely, typically involve users distributed across multiple parties [24, 37]. For instance, Amazon requires regional branches (e.g., America and Asia) to collaboratively estimate the proportion of young customers (aged 18–25) favoring sports equipment priced between \$20–\$50 during Black Friday. Such cross-party analysis is essential for strategic planning (e.g., optimizing marketing campaigns), yet remains unaddressed by existing single-party LDP solutions. This gap stems from two fundamental limitations of multi-party settings, namely prohibitive communication overhead and cross-party statistical heterogeneity.

Intuitively, we can let all users upload their perturbed data using an LDP protocol for frequency estimation (e.g., OUE [28]) to the central server for estimating a global noisy histogram and answering range queries. However, such a method directly leads to **prohibitive communication overhead** on the server side. Specifically, LDP mechanisms often involve encoding a single value into a vector (e.g., with a domain length $|\mathcal{X}| = 2^{20}$), resulting in high communication costs (e.g., up to $O(|\mathcal{U}| \cdot |\mathcal{X}|)$) in the server side where $|\mathcal{U}|$ is the user population size. Although more communication-efficient protocols, such as OLH [28], can be used, their communication overheads remain substantial. Additionally, decoding the randomized data from these mechanisms at the server requires an exhaustive scan of the entire domain $\mathcal{X}$ for each user, which is computationally infeasible for large domains [4, 17, 30].

Fortunately, a cross-party range query can be broken down into several sub-tasks designated to different parties, and the final result naturally emerges from aggregating all the partial results from those parties. For example, a naive solution is to invoke an existing LDP-based range query algorithm (e.g., PrivNUD [27]) in each party, and then the central server answers the queries by aggregating results

†Corresponding author.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

from different parties. However, such a mechanism suffers a lot from the **cross-party statistical heterogeneity** (i.e., non-IID nature of decentralized data). Firstly, the heterogeneity across different parties can lead to skewed (and noisy) internal distributions within different tree nodes or grid cells. As such, the uniform- or linear-distributed assumption made in existing works [7, 26, 27] can lead to large approximation errors when the query intersects with a subdomain. On the other hand, the party-level frequencies of certain ranges estimated by some parties may not contribute positively to estimating the global result. For instance, if some parties have only a small share of certain intervals, these party-level frequencies may be easily overwhelmed by LDP noise.

In this work, we introduce the problem of *federated range queries* under local differential privacy (LDP), which addresses cross-party range queries in decentralized settings. Our solution, the private cubic spline (PrivCS) mechanism, employs a hierarchical interval tree structure to model the underlying data distribution and accurately answer arbitrary range queries. In contrast to prior work [7, 26, 27] that relies on uniform or linear distribution assumptions, PrivCS leverages cubic splines (CS) for distribution fitting. Each spline is parameter-efficient, requiring only a few parameters to represent the distribution within a tree node.

We adopt cubic splines (CS) due to their dual advantages. First, they inherently avoid the Runge phenomenon [19] compared to high-degree interpolation while delivering superior approximation accuracy over low-degree alternatives. PrivCS optimizes CS parameters to simultaneously capture the distribution's geometric properties and minimize expected approximation error. Additionally, data heterogeneity in multi-party settings amplifies instability in party-level estimates (e.g., extreme values) under LDP noise. Naive aggregation of these party-level frequencies may introduce excessive noise. To address this, PrivCS integrates a collaborative frequency filtering (CF) strategy. CF leverages cross-party frequency information to adaptively calibrate estimates using inter-party consistency, selectively prune noise-dominated frequencies, and minimize pruning-induced bias during global aggregation.

The key contributions of our study are summarized as follows:

- This is the first to explore stringent ϵ-LDP solution to answer federated range queries over arbitrary data distribution in a cross-party setting. A novel private cubic spline mechanism is designed to model the underlying data distribution, which eliminates the dependence of assumptions on uniform or linear data distribution.
- We propose the collaborative frequency filtering aggregation that collaboratively post-processes the estimated frequencies by adaptively masking and calibrating certain frequencies to reduce the accumulated errors.
- We conduct comprehensive experiments on real-world and synthetic datasets to confirm the superiority of our mechanism in comparison with existing solutions.

The remainder of this paper is organized as follows. We discuss related works in Section 2. Preliminaries and problem statement are presented in Sections 3 and 4. We detail the proposed mechanism in Section 5, and present the experimental results in Section 6. Finally, we conclude our paper in Section 7.

2 RELATED WORK

Privately answering range queries is a fundamental analytical task that has been extensively studied under DP and LDP. Hay *et al.* [9] propose the first DP hierarchical tree structure combined with a constrained inference mechanism, which was subsequently optimized by Qardaji *et al.* [21]. Xiao *et al.* [32] enhance the hierarchical tree structure using Haar wavelet transforms. Cormode *et al.* [5] introduce a geometric method for privacy budget allocation. Li *et al.* [15] optimize non-uniform domain partitioning. Zhang *et al.* [36] optimize the node decomposition. Huang *et al.* [11] utilize a balanced box-decomposition tree to handle counting queries.

On the other hand, existing studies under LDP can be categorized into tree-based and grid-based methods. Among the tree-based methods, HH [3] and HIO [29] propose basic hierarchical trees with optimal fan-outs. AHEAD [7] improves upon this structure by merging intervals with low frequencies, while PrivNUD [27] tailors the fan-out adaptively for each node. Additionally, DHT [3] optimizes hierarchical trees using Haar wavelet transforms. Regarding grid-based methods, HDG [34] introduces hybrid-dimensional grids, whereas PRISM [31] replaces these simpler grids with prefix-sum cubes. Furthermore, PriPL-Tree [26] approximates the data distribution using piecewise linear functions.

Moreover, there are also other related research topics close to range questions. For instance, McKenna *et al.* [20] introduce a general LDP matrix mechanism for linear queries, it can be specialized for specific range queries, but struggles with efficiently supporting all possible queries. Other studies have addressed a related issue of marginal release under LDP [2, 22, 23, 38]. For example, Zhang *et al.* [38] propose CALM, a framework that reconstructs high-dimensional marginals from low-dimensional estimations. This idea has been widely adopted in multi-dim range query scenarios.

Existing studies under LDP primarily consider the traditional local setting with a single server, neglecting more practical scenarios where users' data are distributed across multiple parties. Furthermore, existing studies rely on simplistic assumptions of uniform or linear internal distributions of subdomains, which may limit their approximation capabilities and consequently result in substantial internal range approximation errors. To the best of our knowledge, PrivCS is the first ϵ-LDP framework specifically developed for answering federated range queries in cross-party settings while addressing the challenges posed by non-IID data distributions.

3 PRELIMINARIES

In this section, we first introduce the definition of LDP in Section 3.1. Then we briefly introduce two classic LDP mechanisms and range query under LDP in Sections 3.2 and 3.3. In Section 3.4, we introduce the cubic spline function. The notations are summarized in Table 1.

3.1 Local Differential Privacy

LDP [12] enables each user to perturb her data locally, without the need for a trusted data curator. It achieves plausible deniability, which is controlled by a privacy budget ϵ. The following provides a formal definition of LDP.

Definition 3.1 (ϵ-LDP). Given a privacy budget $\epsilon > 0$, a randomized mechanism $\mathcal{M} : \mathcal{X} \rightarrow \mathcal{Y}$ provides ϵ-LDP if and only if, for

any two inputs $x, x' \in \mathcal{X}$ and any set of possible outputs $\mathcal{Y}$, the following inequality holds: $\Pr[\mathcal{M}(x) \in \mathcal{Y}] \leq e^{\epsilon} \times \Pr[\mathcal{M}(x') \in \mathcal{Y}]$.

Similar to centralized settings, LDP enjoys the desired property of post-processing [8], which ensures that no privacy leakage occurs when perturbed data undergoes further processing..

THEOREM 3.2. *(Post-Processing) Let $\mathcal{M} : \mathcal{X} \to \mathcal{Y}$ be a mechanism that satisfies ϵ-LDP, and $\mathcal{F} : \mathcal{Y} \to \mathcal{Y}^{\star}$ be an arbitrary randomized mapping. Then $\mathcal{F} \circ \mathcal{M} : \mathcal{X} \to \mathcal{Y}^{\star}$ satisfies ϵ-LDP, where $\circ$ denotes the composition of $\mathcal{F}$ and $\mathcal{M}$.*

3.2 Perturbation Mechanisms

In this subsection, we briefly introduce two classic LDP mechanisms used in this work, namely square-wave (SW) mechanism [18] and optimized unary encoding (OUE) [28].

Square-wave (SW) mechanism. SW is the state-of-the-art LDP mechanism for distribution estimation over numerical data. Given a privacy budget ϵ, SW satisfies ϵ-LDP by taking an input $x \in [0, 1]$ and returning an output $y \in [-b, b+1]$ according to the following probability distribution

$$\Pr[\mathcal{M}(x) = y] = \begin{cases} \frac{e^{\epsilon}}{2be^{\epsilon}+1} & \text{if } |x - y| \leq b, \\ \frac{1}{2be^{\epsilon}+1} & \text{otherwise,} \end{cases}$$

where $b = \frac{\epsilon e^{\epsilon} - e^{\epsilon} + 1}{2e^{\epsilon}(e^{\epsilon} - 1 - \epsilon)}$. The output range will have a higher concentration around the input value as the privacy budget increases, which is beneficial for utility enhancement. SW can produce two estimated noisy histograms, post-processed by Expectation Maximization (EM) or Expectation Maximization with Smoothing (EMS) [18]. The former can capture spiky distributions effectively, while the latter provides a more accurate estimation by smoothing the LDP noise [18, 26]. Thus, although SW outperforms other distribution estimation protocols, its property can likely lead to deeply unstable estimated distributions, depending on the underlying distribution.

Optimized unary encoding (OUE). OUE is one of the state-of-the-art mechanisms for frequency estimation over categorical data. Given a privacy budget ϵ, OUE satisfies ϵ-LDP by encoding an input $x \in \mathcal{X}$ as a $|\mathcal{X}|$-bit one-hot binary vector B, where only the x-th bit is set to 1. It then perturbs each bit of B, and outputs B' by:

$$Pr[B'[x] = 1] = \begin{cases} p^1 = \frac{1}{2} & \text{if } B[x] = 1, \\ p^0 = \frac{1}{e^{\epsilon}+1} & \text{if } B[x] = 0. \end{cases}$$

Let $count_x$ denote the counts of occurrences of B where $B[x] = 1$. The frequency of x is estimated by $f_x^{\text{OUE}} = \frac{count_x/n - p^0}{p^1 - p^0}$, where n signifies the total number of users, and the estimation variance [28] is $Var[f_x^{\text{OUE}}] = \frac{4e^{\epsilon}}{(e^{\epsilon}-1)^2 \cdot n}$. A notable limitation of the OUE protocol is its excessive communication overhead when dealing with significantly large domain sizes. Although its communication-efficient variant, namely the OLH protocol [28] provides an alternative, it incurs substantially higher computational costs, particularly for extremely large domains [4, 30].

Different LDP protocols are subject to distinct practical constraints. In this work, these protocols are generally treated as black boxes, and our primary focus is on designing effective strategies for answering range queries in a federated setting.

Table 1: Notations.

Symbols	Description	Symbols	Description
P_i	The i-th party	$\lvert U_i \rvert$	The user population of P_i
A^{τ}	The τ-th attribute	$\lvert \mathcal{X}^{\tau} \rvert$	The domain size of A^{τ}
$F[j]$	The frequency of j-th bin in histogram F	f^i	A node's party-level frequency estimated in P_i

3.3 Range Query under LDP

The hierarchical tree has emerged as the most efficient data structure for answering 1-dimensional (a.k.a., 1-dim) range queries under LDP. Unlike straightforward methods that require summing up frequencies over all queried bins of a histogram, hierarchical trees significantly reduce query errors by aggregating fewer noisy frequencies. Specifically, hierarchical trees hierarchically decompose the entire domain into disjoint subdomains (a.k.a., intervals) to construct a B-ary tree. In this tree, each node indicates an interval and stores an estimated frequency. As such, a range query can be answered by summing up a few node frequencies, effectively reducing accumulated noise.

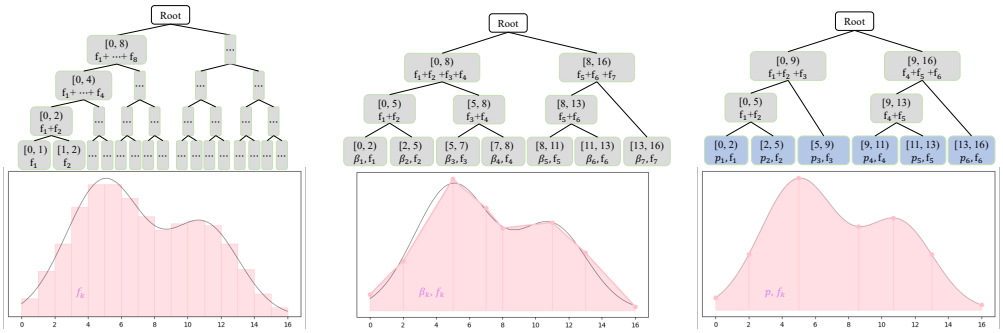
Directly extending hierarchical trees to λ-dimensional (a.k.a., λ-dim, where $\lambda > 1$) range queries is infeasible due to the excessive tree layers and exponentially increasing dimension combinations, which significantly increase the noise error. To answer λ-dim range queries, grid-based approaches are widely adopted. Given m private attributes, a grid-based method estimates the frequencies of 1-dim grids for m attributes and $\binom{m}{2}$ 2-dim grids for attribute pairs. The frequencies of higher-dimensional grids can be approximated using these 1-dim and 2-dim grids through the maximum entropy [38] or the weighted updating techniques [34]. To further reduce error, existing studies optimize granularity [34], use adaptive partition [26], and leverage prefix-sum cubes [31]. Nevertheless, they all rely on a uniform or linear assumption for internal distribution.

3.4 Cubic Spline Function

The cubic spline is extensively used in computer graphics and geometric modeling to obtain curves or motion trajectories that pass through specified points of the plane or 3-D space [1, 6, 13, 14]. Consider a Cartesian coordinate system with an x-axis and a y-axis. To interpolate the internal distribution over a target range $[d_l, d_r)$ on the x-axis, the CS function is defined as follows:

$$\begin{aligned} CS(t) = H_{00}(t) \cdot G[d_l] + H_{10}(t) \cdot (d_r - d_l) \cdot \nabla d_l + \\ H_{01}(t) \cdot G[d_r] + H_{11}(t) \cdot (d_r - d_l) \cdot \nabla d_r, \end{aligned} \tag{1}$$

where $t = (c - d_l)/(d_r - d_l)$ and $c \in [d_l, d_r)$. Here, $G[d_l]$ and $G[d_r]$ denote the function values at d_l and d_r, and ∇d_l, ∇d_r denote their slopes. The functions $H_{00}(t) = 2t^3 - 3t^2 + 1$, $H_{10}(t) = t^3 - 2t^2 + 1$, $H_{01}(t) = -2t^3 + 3t^2$, $H_{11}(t) = t^3 - t^2$ are Hermite basis functions. Compared to other interpolation methods, each approximated interval curve is constructed from two selected adjacent breakpoints and their parameters, independent of other intervals, and can provide a more flexible and accurate approximation by selecting proper parameters. By using CS, we can also answer the same query with fewer nodes. For example, in Figure 1 (c), given a domain $[0, 16)$, a binary CS tree can answer a range query $[0, 7]$ by summing only two node frequencies corresponding to $[0, 5)$ and $[5, 7)$, the latter is approximated using the CS from the node of interval $[5, 9)$.



(a) HT with Uniform Assumptions (b) HT with Linear Assumptions (c) CS tree

Figure 1: An illustration of hierarchical trees with different assumptions (top) and their approximated data distributions (bottom). Left/Middle: a hierarchical tree (HT) assuming a linear/uniform distribution within each leaf. Right: a cubic spline (CS) tree fits the distribution within each leaf using the cubic spline. The black curve represents the ground truth distribution.

4 PROBLEM STATEMENT AND CHALLENGES

4.1 Problem Statement

We consider a federated setting that involves an untrusted central server and a set of untrusted parties $\mathcal{P}$. Each party $P_i \in \mathcal{P}$ consists of a distinct set of users U_i, with no overlap between any two parties. Each user $u_j \in U_i$ holds a private ordinal record comprising m attributes $\{A^1, A^2, \cdots, A^m\}$, represented as $x_j = \langle x_j^1, x_j^2, \ldots, x_j^m \rangle$. The domain of the τ-th attribute A^τ is denoted by $\mathcal{X}^\tau$. Since the parties are not entirely trusted, users will locally perturb their private records via LDP mechanisms and only send noisy records to the parties. Following Definition 3.1, we aim to provide *user-level* ϵ-LDP for each user, who only belongs to a single party. Hence, the perturbed record ensures that an adversary cannot infer any attribute values she owns in the original record with high confidence.

Let $\Gamma \subseteq \{A^1, A^2, \cdots, A^m\}$ be a subset of $\lambda \in [0, m]$ attributes. The objective is to enable the central server to answer *λ-dimensional (λ-dim) federated range queries* over the data distributed across all parties $\mathcal{P}$, while satisfying ϵ-LDP. Formally,

Definition 4.1 (λ-Dim Federated Range Query). Let $Q \triangleq \cap_{\tau=1}^{\lambda} [l_q^\tau, r_q^\tau]$ represent the range over λ dimensions, where $l_q^\tau, r_q^\tau \in \mathcal{X}^\tau$ and $l_q^\tau < r_q^\tau$. The λ-dim federated range query R_Q returns the frequency of records whose queried attribute values fall within the specified ranges, i.e.,

$$R_Q = \frac{\sum_{i=1}^{|\mathcal{P}|} \sum_{j=1}^{|U_i|} \mathbb{I}_{\cap_{\tau=1}^{\lambda} \{l_q^\tau \le x_j^\tau \le r_q^\tau\}}}{\sum_{i=1}^{|\mathcal{P}|} |U_i|}$$

where $\mathbb{I}_y$ denotes the indicator function which returns 1 if the predicate y is true and 0 otherwise.

Remark 1. *Intuitively, a naive solution is to collect perturbed data directly from users of different parties, based on which the central server can answer the federated range queries. However, this approach may incur excessive communication costs. In practice, both the user population and the item domain are typically very large. For example, consider a scenario with* $5,000,000$ *users and a domain size of* $|\mathcal{X}| = 2,000$. *The state-of-the-art frequency estimation mechanism OUE [28] needs to encode each input value into a* $2,000$*-bit vector before sending the perturbed version to the server. This results in a communication cost of approximately* 1×10^{10} *bits, which is prohibitively high and impractical for many real-world applications. Although more communication-efficient protocols, such as OLH, can be used, their communication overheads remain substantial. Additionally, decoding the randomized data from these mechanisms at the server requires an exhaustive scan of the entire domain* $\mathcal{X}$ *for each user, which is computationally infeasible for large domains [4, 17, 30]. Furthermore, such flat methods would introduce significant accumulated noise errors, making the results unreliable for use [3].*

4.2 Straw-Man Solution and Challenges

To answer federated range queries, a straightforward solution is that each party independently applies existing range query mechanisms (e.g., PrivNUD [27] or PriPL-Tree [26]) to construct local hierarchical structures. Then each party uploads its own local structure to the server. The server subsequently queries each received local structure and sums up all the local answers to obtain the overall frequencies. However, this solution overlooks the non-IID problem, posing the following significant challenges for accurately answering federated range queries at the central server.

Large approximation errors under unrealistic assumptions. Data heterogeneity results in varying frequency distributions across parties. The existing LDP mechanisms assume either uniform or linear internal distributions within each node of the hierarchical structures. The unrealistic assumptions lead to large approximation errors when the query intersects with a subdomain. For example, given a hierarchical tree in Figure 1(b), the answer of a query $[4, 10]$ is calculated by summing the frequencies of interval $[5, 8)$ and the partial frequencies $[4, 5)$ and $[5, 10)$ under linear assumptions. That is, they first estimate the ratios of intervals $[4, 5)$ to $[2, 5)$, and $[5, 10)$ to $[5, 11)$, respectively, by assuming linear internal distributions within these intervals. Then, based on these ratios, they approximate the answers accordingly. Such a linear assumption often introduces significant approximation errors, as data distributions rarely conform exactly to linearity in practice. Although we can apply a finer granularity to leaf nodes, it will require more noisy nodes to answer the same query, leading to larger noises. This issue becomes even more pronounced under uniform assumptions, because it provides a worse approximation and needs more nodes to answer.

Complex server-side aggregation under the data heterogeneity and LDP noise. The non-IID nature of data complicates frequency estimation and aggregation across parties. Concretely,

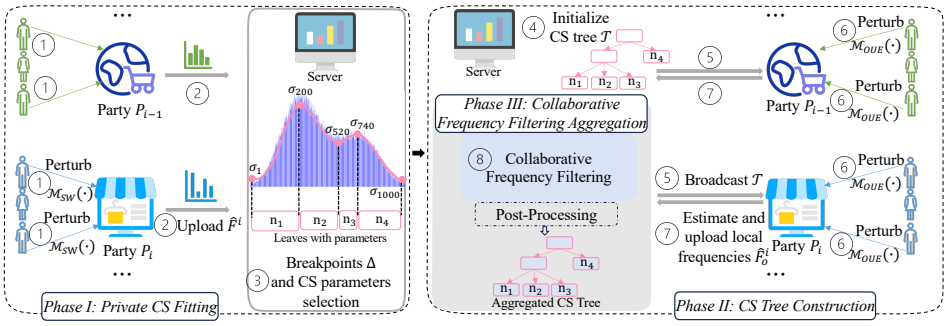


Figure 2: An overview of PrivCS.

local frequencies [1] of certain ranges estimated by some parties may not contribute positively to estimating the global ones. For instance, certain items may be relatively frequently purchased in Amazon Asia but remain unpopular or even absent in Amazon Europe. The frequencies of such items in certain parties do not offer valuable information in aggregation for estimating the overall frequency, since they may be easily overwhelmed by LDP noise. In the following sections, we explore these issues in greater depth and introduce our proposed mechanism designed to mitigate them.

5 PRIVCS: A PRIVATE CUBIC SPLINE MECHANISM

To address the challenges as aforementioned, in this section we propose the private cubic spline (PrivCS) mechanism, which leverages a few cubic splines to capture the underlying data distribution and devises a unified hierarchical tree structure to support federated range queries by integrating the fitting results.

5.1 Design Rationale

The cubic spline has a nice property of accurately approximating skewed distributions as curves via a few breakpoints with some parameters, which thus avoids a uniform or a linear assumption. Furthermore, CS is capable of preventing the Runge phenomenon compared to high-degree interpolation, while providing more accurate approximations than low-degree methods [19]. This inspires us to design PrivCS that fits the data distribution via cubic splines for answering range queries over arbitrary distributions. On the other hand, since the data heterogeneity may result in varying party-level distributions, fitting them independently fails to capture the overall distribution geometry. Therefore, to address this issue and facilitate range query processing, PrivCS first fits the overall distribution and then constructs a globally optimal hierarchical tree, referred to as the CS tree. Each leaf node represents an interval and stores the corresponding CS parameters and interval frequency. Non-leaf nodes represent their intervals by combining those of their children and store only the frequencies. In contrast to other hierarchical trees [7, 26, 27], CS tree enables us to use fewer nodes (thus less LDP noise [26]) and more accurate approximations to model the skewed distribution, making it well-suited for federated settings.

Moreover, some party-level frequency estimates can be easily overwhelmed by LDP noise. Simply aggregating such estimates of the same interval to compute its overall frequency may introduce excessive noise in federated settings. To this end, we introduce the collaborative frequency filtering (CF) aggregation. In this aggregation, we leverage some valuable frequency information across parties for the same intervals to adaptively calibrate the frequency estimates for each node, and selectively prune the local frequencies of certain nodes that exceed the pruning threshold by analyzing the pruning error, thereby reducing the cumulative LDP noise.

5.2 An Overview of PrivCS

The above design rationale leads to our private cubic spline (PrivCS) mechanism. Figure 2 shows an overview of the mechanism, which consists of three phases, namely private CS fitting, CS tree construction, and collaborative frequency filtering aggregation.

Phase I: Private CS fitting. Our goal is to produce the optimal CS fitting using a set of curves tailored to model interval-wise distributions. Each curve models the internal distribution of an interval, which includes all the bucketized values between the starting and ending bins (a.k.a *breakpoints*). For example, the interval $[2, 5)$ includes $\{2, 3, 4\}$ between breakpoints 2 and 5. Eventually, the fitting should capture the entire distribution geometry by selecting the optimal parameters for each interval. Unfortunately, the data distribution is invisible. Thus, to gain fundamental insight into the distribution, we first estimate a global histogram under LDP as a reference. Specifically, each party selects a small fraction α of its users to estimate the histogram $\hat{F}^i$ using SW $\mathcal{M}_{SW}(\cdot)$ (without post-process) with privacy budget ϵ (step ①), then uploads its estimated histogram to the server (step ②) for aggregation. The server then fits the distribution by selecting breakpoints and parameters. The breakpoints are determined based on the geometry of the histogram to divide the entire domain into disjoint intervals, which will initialize the leaves of the CS tree. Then the server selects the optimal parameters by minimizing the expected approximation errors when fitting each leaf's internal distribution (step ③).

Phase II: CS tree construction. We construct the CS tree in this phase. The server first initializes the tree from bottom to top based on the estimated leaves, and broadcasts the tree with necessary parameters to each party for further estimation (steps ④–⑤). Upon receiving them, each party divides the remaining users into several groups (depending on the tree structure) to estimate frequencies of different intervals. Users from each group in Party P_i perturb and upload their data using OUE with privacy budget ϵ to P_i, denoted by $\mathcal{M}_{OUE}(\cdot)$ (step ⑥). Party P_i then aggregates the sanitized data from these groups to estimate the frequencies of the corresponding nodes (i.e., intervals) in the CS tree. Finally, the parties transmit their estimated frequencies back to the server for aggregation (step ⑦).

Phase III: Collaborative frequency filtering aggregation. The proposed CF aggregation (step ⑧) is applied to *calibrate*, *prune*, and *aggregate* the local frequencies of each node across different parties. For each node's interval, we have two frequency estimates: one from the histogram estimation phase (at the beginning of Phase I), and

[1] For ease of understanding, we refer to party-level frequencies estimated in a single party P_i as local frequencies.

the other from the frequency estimation phase (Phase II). These two estimates are intrinsically related as they estimate the same interval frequency via different methods with different purposes.

The key idea is to leverage these relationships to refine local frequency estimates and reduce cumulative errors. Therefore, the CF strategy utilizes the internal relationships to calibrate the frequency estimates for each node while selectively pruning some local frequencies. The pruning is only applied to local frequencies where the pruning error is smaller than not pruning. For each node, the server aggregates all its refined local frequencies (by employing CF) from different parties to obtain the overall interval frequency. Finally, the server applies a post-processing step to further enhance overall frequency consistency, thus producing an optimal CS tree.

Answering 1-dim federated range queries. Given a federated range query $Q = [l_q, r_q]$, the answer is obtained by summing up the node frequencies that are fully covered by the range $[l_q, r_q]$ but whose parents are not entirely within this range, along with partial frequencies from leaf nodes that overlap but are not completely within $[l_q, r_q]$. For instance, given the CS tree shown in Figure 1(c), the answer to the query $[4, 10]$ is calculated by summing up the frequency of the interval $[5, 9)$, a partial frequency of the interval $[2, 5)$, and a partial frequency of the interval $[9, 11)$. The proportions for intervals $[2, 5)$ and $[9, 11)$ are computed based on the ratios of the overlap integrals of the corresponding nodes; specifically, $\pi_2 = \int_4^5 CS_2(t)dt / \int_2^5 CS_2(t)dt$ and $\pi_4 = \int_9^{10} CS_4(t)dt / \int_9^{11} CS_4(t)dt$, where $CS_2(\cdot)$ and $CS_4(\cdot)$ represent the corresponding CS functions with their parameters. Thus, the final answer to Q is computed as $\pi_2\hat{f}_2 + \hat{f}_3 + \pi_4\hat{f}_4$, where $\hat{f}_k$ denotes the noisy frequency of node n_k.

5.3 Private CS Fitting

In this subsection, we introduce our method for establishing a global interval partition, which serves as the foundation for constructing the CS tree in the subsequent phase. The CS fitting characterizes each interval by a pair of neighboring *breakpoints*, along with their associated frequencies and derivatives. As illustrated in Figure 1(c), the breakpoint set $\Delta = \{0, 2, 5, 9, 11, 13, 16\}$ determines six intervals: $[0, 2), [2, 5) \ldots, [13, 16)$. For each interval $[l, r)$, we then optimize the cubic spline function by modifying Equation (1) as follows:

$$\begin{aligned} CS(t) = H_{00}(t) \cdot \hat{F}_s[l] + H_{10}(t) \cdot (r - l) \cdot \nabla\sigma_l + \\ H_{01}(t) \cdot \hat{F}_s[r] + H_{11}(t) \cdot (r - l) \cdot \nabla\sigma_r, \end{aligned} \tag{2}$$

where $t = (c-l)/(r-l)$ and $c \in [l, r)$, $\hat{F}_s[l]$ and $\nabla\sigma_l$ (resp. $\hat{F}_s[r]$ and $\nabla\sigma_r$) represent the noisy frequency and the noisy first derivative at l-th bin σ_l (resp. r-th bin σ_r).

The private CS fitting includes three essential steps: (1) the overall histogram estimation, (2) the breakpoints selection, and (3) the CS parameter optimization. The first step is to provide a rough reference for the subsequent breakpoints and parameters selections, which are the main challenges in this phase. The breakpoints selection is non-trivial, as it requires balancing the accurate approximation of the distribution's geometry with the accumulated LDP noise. Furthermore, directly optimizing the CS parameters based on the estimated data distribution can introduce excessive noise, particularly when the privacy budget is constrained. We will address these technical challenges as follows.

Algorithm 1: Private CS Fitting

Input: Parties $\mathcal{P}$, dividing ratio α, and privacy budget ϵ
Output: The sorted breakpoint set Δ^R and corresponding parameters
`// Party side`
1 **for** $P_i \in \mathcal{P}$ **do**
2 $\quad P_i$ requires a small part of its users $\alpha \cdot U^i$ to estimate the noisy histogram $\hat{F}^i$ via SW with privacy budget ϵ;
3 $\quad P_i$ uploads $\hat{F}^i$ to server;
`// Server side`
4 The server aggregates histograms received from each party, then applies EMS and EM post-processing steps to obtain the overall histograms $\hat{F}_s$ and $\hat{F}_m$;
5 The server solves Equation (3) to obtain the breakpoint set Δ;
6 Sort the breakpoints in Δ in a ascending order by the bin index as Δ^R;
7 **for** $1 \leq idx \leq |\Delta^R| - 1$ **do**
8 $\quad \sigma_l = \Delta^R[idx]$ and $\sigma_r = \Delta^R[idx + 1]$;
9 $\quad$ Solve Equation (4) with σ_l and σ_r to obtain the optimal parameters of each leaf node;
10 **return** Δ^R and corresponding parameters;

5.3.1 Histogram estimation. To obtain the global histogram across all parties, as outlined in lines 1–3 of Algorithm 1, each party $P_i \in \mathcal{P}$ uploads its noisy histogram $\hat{F}^i$ estimated by SW [18] to the server. The server then aggregates these histograms and applies the EM and EMS optimization steps of SW to obtain $\hat{F}_m$ and $\hat{F}_s$ (line 4).

5.3.2 Breakpoints selection. The breakpoints define the interval partition. Increasing the number of breakpoints yields finer-grained approximations of the underlying data distribution but also results in more leaves (thus more layers), which introduce more LDP noise [27]. To strike a balance between capturing the distribution's geometry and minimizing noise accumulation, we heuristically select breakpoints based on the approximated first and second derivatives of the bins, leveraging the observation that bins with near-zero derivatives exhibit greater stability under LDP noise. More precisely, let σ_j denote the j-th bin of the estimated histogram $\hat{F}_s$ for the attribute domain $\mathcal{X}_\tau$, the approximated first and second derivatives at σ_j are calculated as follows:

$$\nabla\sigma_j = \frac{\hat{F}_s[j+1] - \hat{F}_s[j-1]}{2},$$
$$\nabla^2\sigma_j = \hat{F}_s[j-1] - 2\hat{F}_s[j] + \hat{F}_s[j+1].$$

The breakpoint set Δ comprises all bins where both derivatives vanish within a δ-tolerance:

$$\Delta = \{\sigma_j | \nabla\sigma_j \in [-\delta, \delta], \nabla^2\sigma_j \in [-\delta, \delta], \sigma_j \in \mathcal{X}_\tau\}. \tag{3}$$

Note that δ could be set as a small positive value inversely proportional to the total user population $|U| = \sum_{i=1}^{|\mathcal{P}|} |U^i|$. For the first and last bins, we use backward and forward differences instead. The server sorts Δ in ascending order. Each interval is defined by a pair of adjacent breakpoints in this ordered set.

5.3.3 Parameters selection. In this subsection, we explain how the server selects the optimal parameters for a given leaf's interval $[l, r)$. Recall that in Equation (2), the optimal CS function $CS^*(\cdot)$ for $[l, r)$ depends on four key parameters, i.e., frequencies and the first derivatives at the bins σ_l and σ_r. While one could directly use the noisy estimates $\hat{F}_s[l]$, $\hat{F}_s[r]$, $\nabla\sigma_l$, and $\nabla\sigma_r$ obtained from the noisy histogram $\hat{F}_s$, this approach would propagate significant LDP noise

into the final results. To achieve a balance between approximation error and computational efficiency, we choose to refine only the first derivatives $\nabla\sigma_l$ and $\nabla\sigma_r$, while retaining the noisy frequency estimates $\hat{F}_s[l]$ and $\hat{F}_s[r]$. This choice is supported by empirical observation showing that frequencies exhibit greater robustness to noise perturbations compared to the first derivative.

In a nutshell, our objective is to determine the optimal $\nabla\sigma_l^*$ and $\nabla\sigma_r^*$ that minimize the expected approximation error for all possible range queries $Q_\cap \triangleq \{[l_q, r_q] | [l, r) \cap [l_q, r_q] \neq \emptyset\}$ intersecting the interval $[l, r)$, as the CS function (or uniform/linear assumption) only works when intersecting. Note that $Q_\cap$ are derived by enumerating all the possibly intersected queries with uniform arrival probabilities [27]. Formally, we aim to select

$$\nabla\sigma_l^*, \nabla\sigma_r^* = \underset{\nabla\sigma_l, \nabla\sigma_r}{\arg\min} \frac{1}{|Q_\cap|} \sum_{[l_q, r_q] \in Q_\cap} \sum_{\hat{F} \in \{\hat{F}_s, \hat{F}_m\}} \eta(l, r, l_q, r_q, CS(\cdot), \hat{F}), \tag{4}$$

$$\text{such that} \quad CS^*(t) = H_{00}(t) \cdot \hat{F}_s[l] + H_{10}(t) \cdot (r - l) \cdot \nabla\sigma_l^* + H_{01}(t) \cdot \hat{F}_s[r] + H_{11}(t) \cdot (r - l) \cdot \nabla\sigma_r^*,$$

where $\eta(\cdot)$ denotes the error function. Note that the approximation error is manifested in answering the partial frequency of the intersected subinterval $[l_q, r_q] \cap [l, r)$, as this frequency is computed by multiplying the *frequency proportion* (approximated by the CS function or other assumptions) with the interval frequency of $[l, r)$. For example, given a node n_k with the interval $[2, 5)$, the partial frequency of $[4, 5)$ is $\pi_k f_k$, where π_k is the frequency proportion of $[4, 5)$ and f_k is the interval frequency of n_k. Therefore, we define $\eta(\cdot)$ to measure the difference between the frequency proportions of $[l_q, r_q] \cap [l, r)$ computed by $CS(\cdot)$ (with parameters $\nabla\sigma_l$ and $\nabla\sigma_r$) and that estimated from the reference histogram, i.e.,

$$\eta(l, r, l_q, r_q, CS(\cdot), \hat{F}) = [\varphi(l, r, l_q, r_q, CS(\cdot)) - \omega(l, r, l_q, r_q, \hat{F})]^2,$$

$$\varphi(l, r, l_q, r_q, CS) = \frac{\int_{l_\cap}^{r_\cap} CS(t)dt}{\int_l^r CS(t)dt},$$

$$\omega(l, r, l_q, r_q, \hat{F}) = \frac{\sum_{t=l_\cap}^{r_\cap} \hat{F}[t]}{\sum_{t=l}^{r} \hat{F}[t]},$$

where $l_\cap = \max\{l_q, l\}$ and $r_\cap = \min\{r_q, r\}$. Here $\varphi(\cdot)$ computes the definite integral of the CS function $CS(\cdot)$, whereas $\omega(\cdot)$ computes the discrete sum of the histogram $\hat{F}$ over the corresponding bins. To accelerate the optimization of Equation (4), we search $\nabla\sigma_l$ (resp. $\nabla\sigma_r$) in the range of $[\nabla\sigma_l - e^{-\epsilon}/2, \nabla\sigma_l + e^{-\epsilon}/2]$ (resp. $[\nabla\sigma_r - e^{-\epsilon}/2, \nabla\sigma_r + e^{-\epsilon}/2]$), where the term $e^{-\epsilon}/2$ is introduced to narrow down the searching space as ϵ increases, since a larger ϵ results in more accurate estimations, and vice versa.

Finally, the server searches for the optimal parameters by solving Equation (4) for each interval defined by the breakpoints in Δ. These intervals, along with their corresponding optimal parameters, are stored as the leaves of the CS tree (lines 5–9 of Algorithm 1).

5.4 CS Tree Construction

Given the leaves with optimal parameters, the next step is to construct the CS tree, which involves three steps: CS tree initialization, user allocation, and frequency estimation.

Algorithm 2: CS tree Construction

Input: Parties $\mathcal{P}$, dividing ratio α, breakpoint set Δ^R, and privacy budget ϵ
Output: All the estimated frequencies $\{\hat{F}_o^1, \hat{F}_o^2, \cdots, \hat{F}_o^{|\mathcal{P}|}\}$

```
// Server side: CS tree initialization
1  Construct a balanced binary tree T based on Δ^R;
2  for node n_k in postorder traversal do
3      Compute Err and Err' for T with n_k and w/o n_k via Equation (5);
4      if Err > Err' then
5          Remove n_k;
6  The server broadcasts the reduced T to each party;
// Party side: User allocation and node frequency estimation
7  for P_i ∈ P do
8      P_i assigns available users U_0^i = U^i − α · U^i to the root n_0;
9      for each node n_k of T in level order traversal do
10         if n_k is root then
11             Set U_k^{i'} = ∅;
12         else
13             Compute h_k, the height of the subtree rooted at n_k;
14             Sample ⌈|U_k^i|/h_k⌉ users from U_k as U_k^{i'};
15             Allocate U_k^{i'} to n_k for frequency estimation;
16         for n_c ∈ children(n_k) do
17             Assign U_k^i − U_k^{i'} to n_c;
18     for each user u_j ∈ U_0^i do
19         Assign intervals of nodes N_i = {n_k | u_j ∈ U_k^{i'}} to u_j;
20         u_j uploads the sanitized value perturbed by OUE with ε to P_i;
21     P_i aggregates each node's frequency and uploads the estimated
       frequencies F̂_o^i to the server.
22 return All the estimated frequencies {F̂_o^1, F̂_o^2, ···, F̂_o^{|P|}};
```

5.4.1 CS tree initialization. At first, the server constructs a default binary tree from bottom to top using these leaves (line 1 in Algorithm 2). Then, the server adaptively reduces non-leaf nodes. The reduction of a node involves deleting it and linking its all child nodes to its parent node. Such a reduction occurs if the average error of deleting a node is smaller than the error from preserving it [26, 27]. Specifically, let $\mathcal{X}_\tau$ denote the domain of attribute τ. We evaluate the average error by calculating the expected accumulated noise from nodes intersected by all possible query ranges,

$$\sum_{n_k \in \mathcal{N}} w_k \cdot Err(\hat{f}_k), \quad w_k = \frac{l_k \cdot (|\mathcal{X}_\tau| - r_k + 1) - l_p \cdot (|\mathcal{X}_\tau| - r_p + 1)}{(|\mathcal{X}_\tau| + 1)|\mathcal{X}_\tau|/2}, \tag{5}$$

where $[l_k, r_k]$ (resp. $[l_p, r_p]$) is the interval stored of node n_k (resp. its parent node n_p) and w_k denotes the probability of n_k being involved in queries. Since we will use OUE for frequency estimation, whose variance is inversely proportional to the allocated user population, we approximate $Err(\hat{f}_k)$ by $1/\alpha_k$ for convenience, where α_k is the user allocation ratio for n_k [26, 27]. Once the reduction is completed, the server broadcasts the initialized CS tree to each party for user allocation (lines 2–6 of Algorithm 2).

5.4.2 User allocation. Following the previous study [26], each party P^i distributes its users along relative paths in the CS tree. All users are initially assigned to the root node, which serves as the starting point and does not require frequency estimation. For each non-root node n_k (processed in level-order), it receives $\lceil |U_k^i|/h_k \rceil$ users from its parent, denoted as $U_k^{i'}$. Here, U_k^i is the unassigned user set of n_k's parent, and h_k denotes the height of the sub-tree rooted at n_k. For the leaf nodes, they are all assigned $U_k^i \setminus U_k^{i'}$ users.

5.4.3 Frequency estimation. Each party requires its users to perturb and upload their sanitized data, to estimate frequencies of all nodes by OUE with privacy budget ϵ. Note that each user only participates in the assigned nodes' frequency estimations. Then, each party sends all the nodes' local frequencies to the server for aggregation (lines 7–21 of Algorithm 2).

5.5 Collaborative Frequency Filtering Aggregation

The most straightforward approach to compute a node's overall interval frequency is to aggregate the noisy local frequencies reported by all parties. However, this method may introduce significant noise in cross-party settings due to two key factors: 1) a small privacy budget ϵ potentially leads to some extremely large or small local frequency estimations, and 2) certain local frequencies are particularly susceptible to being overwhelmed by LDP noises, especially when these parties have only a small share of specific intervals. To overcome these issues, we propose a *collaborative frequency filtering (CF) aggregation* method consisting of three key steps: local frequency calibration, local frequency pruning, and local frequency aggregation. The CF method operates on all node-level frequency estimates within the CS tree.

5.5.1 Local frequency calibration. The calibration step aims to correct extreme local frequency estimates of each node reported by certain parties, thereby reducing aggregation errors. Consider an arbitrary node in the CS tree, its perturbed local frequency estimated by party P_i via OUE in Phase II is denoted as $\hat{f}_o^i$. For the same node, we can also obtain its local frequency $\hat{f}_s^i$ and the overall frequency $\hat{f}_s$ estimated by SW in Phase I. The former is the summation of all bins' frequencies within the node interval in the noisy histogram $\hat{F}_s^i$ estimated by P_i in Phase I, while the latter is the summation of the same bins' frequencies of the overall noisy histogram $\hat{F}_s$. Although the frequencies estimated by SW contain noise, they still provide a rough reference for the relationship between the P_i level and cross-party level frequencies. Therefore, instead of using the unstable frequencies themselves, we leverage the ratio between $\hat{f}_s^i$ and $\hat{f}_s$ to constrain and refine the OUE estimates as follows:

$$\hat{f}_o^i = \hat{f}_o^i \cdot (1+\mu), \mu = \begin{cases} -\frac{\hat{f}_o^i - b}{e^\epsilon \hat{f}_o^i} & \text{if } \hat{f}_o^i > b, \\ \frac{a - \hat{f}_o^i}{e^\epsilon \hat{f}_o^i} & \text{if } \hat{f}_o^i < a, \end{cases} \tag{6}$$

$$a = \hat{f}_s^i \cdot (1-\beta), b = \hat{f}_s^i \cdot (1+\beta), \beta = \frac{|\hat{f}_s^i - \hat{f}_s|}{\hat{f}_s}.$$

The above refinement calibrates estimated local frequencies that fall outside the feasible range $[a, b]$, which is derived from the offset β between $\hat{f}_s^i$ and $\hat{f}_s$. The β can be treated as a frequency measurement of the heterogeneity between the P_i-level distribution and the overall distribution of the current interval. That is, for $\hat{f}_o^i$ that falls outside this range, we mark it as an infeasible estimate that is necessary to be refined, since it exceeds the frequency bound $[a, b]$ under the current heterogeneity. Therefore, we use the μ to adjust the infeasible frequencies to be closer to $[a, b]$. The μ consists of the difference ratio between $b - \hat{f}_o^i$ (resp. $a - \hat{f}_o^i$) and $\hat{f}_o^i$ when $\hat{f}_o^i > b$ (resp. $\hat{f}_o^i < a$) and the decay factor $e^{-\epsilon}$, which weakens μ as ϵ increases, since the OUE estimates become more accurate.

Algorithm 3: Collaborative Frequency Filtering Aggregation

Input: Parties $\mathcal{P}$, overall histogram $\hat{F}_s$ estimated by SW, all the estimated frequencies $\{\hat{F}_o^1, \hat{F}_o^2, \cdots, \hat{F}_o^{|\mathcal{P}|}\}$, and the overall CS tree $\mathcal{T}$
Output: The optimal CS tree $\mathcal{T}$
`// Server side`
1 The server calculates the optimal threshold ξ^* with corresponding ξ_1^* and ξ_2^* via Equation (9);
2 **for** *each node n of $\mathcal{T}$ in post-order traversal* **do**
3 Calculate the overall interval frequency $\hat{f}_s$ of node n estimated in Phase I by summing its corresponding bins' frequencies in $\hat{F}_s$ up;
4 **for** $1 \le i \le |\mathcal{P}|$ **do**
5 Calibrate each $\hat{f}_o^i$ of node n estimated by P_i by Equation (6), where $\hat{f}_o^i$ is the OUE-estimated local frequency of node n in $\hat{F}_o^i$;
6 **if** *threshold $\xi^* > 0$ and $\hat{f}_o^i / \hat{f}_s \le \xi_1^*$ and $\hat{f}_s \le \xi_2^*$ and $\hat{f}_o^i \le \xi^*$* **then**
7 $\hat{f}_o^i = 0$;
8 Compute the overall OUE-estimated frequency $\hat{f}_o^i = \sum_{1 \le i \le |\mathcal{P}|} |U^i| \cdot \hat{f}_o^i / |U|$, where $|U|$ (resp. $|U^i|$) denotes the overall (resp. i-th party's) user population;
9 Employ the weighted averaging and frequency consistency procedures for $\mathcal{T}$ to enhance the utility and consistency;
10 **return** $\mathcal{T}$;

5.5.2 Local frequency pruning. After calibrating the local frequencies of all nodes in the CS tree (lines 5–7 of Algorithm 3), we must decide whether to accept or prune these calibrated local estimates during the global aggregation. This decision presents a key trade-off: acceptance introduces additional LDP noise, while rejection may result in the loss of meaningful frequency information. To balance this trade-off, we introduce local frequency pruning, which rejects local frequencies smaller than a threshold. We analytically determine the optimal *pruning threshold* ξ^* by comparing the expected errors of both choices. For a node's noisy local frequency $\hat{f}_o^i$ estimated by P_i and a given pruning threshold ξ, the estimation error κ under ξ may come from two situations: 1) the ground truth frequency f_o^i is larger than ξ but $\hat{f}_o^i$ is smaller than ξ, and 2) both frequencies are smaller than ξ. Different situations result in different κ. Thus, we take both situations into account by simulating the perturbation for calculating their errors.

The case of acceptance. Given a pruning threshold ξ, we compute the average error when accepting local frequencies smaller than ξ for aggregation as follows:

$$\begin{aligned} Err_1(\xi) &= \mathbb{E}[Pr(\hat{f}_o^i \le \xi) \cdot \kappa] \\ &= \mathbb{E}[Pr(\hat{f}_o^i \le \xi | f_o^i) \cdot Pr(f_o^i \ge \xi) \cdot \kappa] + \mathbb{E}[Pr(\hat{f}_o^i \le \xi | f_o^i) \cdot Pr(f_o^i \le \xi) \cdot \kappa] \\ &\approx \sum_{v^+ \in \mathcal{V}^+} \mathbb{E}[Pr(\hat{v}^+ \le \xi | v^+) \cdot (\xi - \hat{v}^+)] + \sum_{v^- \in \mathcal{V}^-} \mathbb{E}[Pr(\hat{v}^- \le \xi | v^-) \cdot |\hat{v}^- - v^-|] \\ &\approx \sum_{y' \in \mathcal{Y}'} [\sum_{v^+ \in \mathcal{V}^+} Pr(y' | v^+) \cdot (\xi - y') + \sum_{v^- \in \mathcal{V}^-} Pr(y' | v^-) \cdot |y' - v^-|] \end{aligned}$$

$$\leq \sum_{y' \in \mathcal{Y}'} [\sum_{v^+ \in \mathcal{V}^+} (\xi - y') \cdot \phi(\frac{y' - v^+}{Var}) + \sum_{v^- \in \mathcal{V}^-} |y' - v^-| \cdot \phi(\frac{y' - v^-}{Var})], \tag{7}$$

where f_o^i (resp. $\hat{f}_o^i$) denotes the unknown true local frequency (resp. its perturbed version), and κ is the error introduced for $\hat{f}_o^i$ under ξ. To satisfy LDP, we sample $v \in \mathcal{V}$ from the noisy histogram $\hat{F}_s$ estimated by SW with EMS in Phase I, treating it as the "true" frequency, while using an independent $y' \in \mathcal{Y}' = \{0.01, 0.02, \cdots, \xi\}$ as the worst-case "noisy" frequency appearing uniformly. We use $\mathcal{V}^+ = \{v \mid v \geq \xi, v \in \mathcal{V}\}$ to denote the set of "noisy" frequencies larger than ξ, and define $\mathcal{V}^- = \mathcal{V} \setminus \mathcal{V}^+$ as the remaining values. Here, $\phi(\cdot)$ denotes the CDF of the Gaussian distribution and Var is the variance of OUE, since a local frequency estimated by OUE can be approximated by a Gaussian distribution [30]. The first term in Equation (7) simulates the acceptance error for the first case in aggregation, and we bound this error as $\kappa = \xi - y'$ to prevent it from dominating the equation. The second term represents the error in the second case, where the error is given by $\kappa = |y' - v^-|$. We employ the CDF of a Gaussian distribution $\phi(\cdot)$ to compute an upper bound for $Pr(y'|v^+)$ or $Pr(y'|v^-)$, which corresponds to the probability that the perturbed y' is smaller than v^+ or v^-.

The case of rejection. Similar to the case of acceptance, we compute the average error for rejection (namely pruning) as follows:

$$\begin{aligned} Err_0(\xi) &\approx \sum_{y' \in \mathcal{Y}'} [\sum_{v^+ \in \mathcal{V}^+} Pr(y'|v^+) \cdot \xi + \sum_{v^- \in \mathcal{V}^-} Pr(y'|v^-) \cdot v^-] \\ &\leq \sum_{y' \in \mathcal{Y}'} [\sum_{v^+ \in \mathcal{V}^+} \xi \cdot \phi(\frac{y' - v^+}{Var}) + \sum_{v^- \in \mathcal{V}^-} v^- \cdot \phi(\frac{y' - v^-}{Var})]. \end{aligned} \tag{8}$$

The error is ξ if we prune the "noisy" estimate y', which is smaller than ξ when $v^+ > \xi$, while the error is v^- when v^- is smaller than ξ. In the implementation, we simply approximate Equations (7) and (8) by sampling 100 values as $\mathcal{V}$ and computing the expectations.

We then detail how to search for the optimal threshold ξ^*. Since small local frequencies that have a small share of their overall frequencies are more likely to be overwhelmed by the LDP noise, instead of directly searching ξ^*, we decompose the selection of the optimal ξ^* into two searching conditions: 1) the local frequency importance ξ_1, measured by the ratio between the $\hat{f}_o^i$ and $\hat{f}_s$, and 2) the overall frequency importance ξ_2, i.e., the overall frequency in Phase I. That is, we set $\xi = \xi_1 \cdot \xi_2$, so that the searching space of ξ is composed of $\xi_1 \in [0, e^{-\epsilon}/2]$ and $\xi_2 \in [\xi_1, e^{-\epsilon}/2]$. Only the local frequency importance and overall frequencies of intervals that are smaller than the optimal ξ_1^* and ξ_2^* are considered to be pruned. The optimal $\xi^* = \xi_1^* \cdot \xi_2^*$ is selected as follows:

$$\begin{aligned} \xi_1^*, \xi_2^* &= \arg\min_{\xi_1, \xi_2} Err_0(\xi), \\ \text{s.t.} \quad & Err_0(\xi) < Err_1(\xi), \xi = \xi_1 \cdot \xi_2. \end{aligned} \tag{9}$$

Therefore, we select the optimal threshold ξ^* that leads to the minimum rejection error if there exists a candidate threshold whose rejection error is smaller than the acceptance error and use the same ξ^* to prune all the nodes' local frequencies that satisfied the searching conditions; otherwise, we do not apply pruning (line 1 and lines 6–7 of Algorithm 3).

Table 2: Communication costs.

	PrivNUD or AHEAD	PriPL-Tree	OUE	PrivCS
Communication	$O(b \cdot \lvert\mathcal{N}\rvert \cdot \lvert\mathcal{P}\rvert)$	$O(b \cdot \lvert\mathcal{P}\rvert \cdot (\lvert\mathcal{N}\rvert + \lvert\mathcal{X}\rvert))$	$O(\lvert\mathcal{X}\rvert \cdot \lvert\mathcal{U}\rvert)$	$O(b \cdot \lvert\mathcal{P}\rvert \cdot (\lvert\mathcal{N}\rvert + \lvert\mathcal{X}\rvert))$

5.5.3 Local frequency aggregation. The CF enables us to refine and prune certain local frequencies by considering both local and global frequency information. Subsequently, we populate the overall CS tree with these calibrated frequencies. For each node, we compute and store the overall frequency. For instance, given a node's frequency $\hat{f}^0$ from party P_0 with $|U^0|$ users and $\hat{f}^1$ from party P_1 with $|U^1|$ users, the overall frequency of this node is $(\hat{f}^0 \cdot |U^0| + \hat{f}^1 \cdot |U^1|)/(|U^0| + |U^1|)$. However, frequency inconsistency issues may arise. For example, the inconsistencies between a parent node's frequency and the sum of its child nodes, as well as discrepancies between the frequencies of leaf nodes estimated by SW and OUE. To mitigate these issues, we employ weighted averaging and frequency consistency post-processing (line 9 of Algorithm 3), following the previous studies [26, 27]. We provide the details in Appendix A.1.

5.6 Privacy and Communication Cost Analysis

5.6.1 Privacy analysis. In PrivCS, each user uploads its noisy data perturbed by an LDP mechanism (i.e., SW or OUE) to its respective party only once, either in Phase I or Phase II. All subsequent computations performed by the parties or the central server rely solely on these sanitized inputs. Since LDP guarantees are preserved under post-processing, this ensures that the entire PrivCS mechanism satisfies ϵ-LDP, as shown in Theorem 5.1. Due to the page limit, we defer a detailed proof to Appendix A.2.

Theorem 5.1. *The PrivCS mechanism satisfies ϵ-LDP.*

5.6.2 Communication costs. Let $|\mathcal{N}|$ denote the total number of nodes in the hierarchical tree, $|\mathcal{X}|$ be the domain size of the attribute, $|\mathcal{P}|$ be the number of parties, and each pair of a node/interval and its frequency consumes b bits. PrivCS incurs communication costs in two phases. In Phase I, each party uploads the noisy histogram to the server, leading to the $b \cdot |\mathcal{P}| \cdot |\mathcal{X}|$. In Phase II, each party then uploads the OUE estimates of all nodes in the CS tree to the server for the aggregation, introducing the $|\mathcal{N}|$ term. The aggregation phase does not require additional communication. The total communication cost of PrivCS is therefore $O(b \cdot |\mathcal{P}| \cdot (|\mathcal{N}| + |\mathcal{X}|))$.

For comparison, we consider adapting existing 1-dim baselines to federated settings. Without loss of generality, we assume that all methods employ identical hierarchical tree structures (and thus have the same total number of nodes). PrivNUD and AHEAD incur communication costs of $O(b \cdot |\mathcal{N}| \cdot |\mathcal{P}|)$, since each party in $\mathcal{P}$ uploads all $|\mathcal{N}|$ nodes of its estimated hierarchical tree and corresponding frequencies to the server. PriPL-Tree incurs the same communication costs as PrivCS, since both require distribution information, which introduces the $|\mathcal{X}|$ term. As for the method of directly uploading the perturbed user data (e.g., via OUE) to the central server, it results in prohibitive communication costs that increase linearly with the total user population $|\mathcal{U}|$. All results are

Table 3: Datasets.

Dataset	Description	# Users	# Total Users
Amazon	comments and ratings	415,271	1,536,684
		877,368	
		244,045	
Instacart	grocery orders	895,832	3,346,083
		844,162	
		803,873	
		802,216	
Tokyo	check-ins	300,203	382,409
		82,206	
Gaussian	Gaussian $(0, \sigma^2)$, $\sigma^2 \in [1, 10]$	50,000	300,000
		100,000	
		150,000	
Zipf	Zipf (α), $\alpha \in [1.01, 1.5]$	9,746	80,677
		17,900	
		23,824	
		29,207	
Hybrid	Gaussian $(0, \sigma^2)$, $\sigma^2 \in [1, 1.5]$, Zipf (1.01), Gamma (1.5, 1000), and Cauchy (1.0)	30,000	909,000
		29,000	
		100,000	
		100,000	
		150,000	
		500,000	

summarized in Table 2. It is worth noting that while communication overheads are comparable, our PrivCS mechanism consistently achieves superior utility, which will be elaborated in Section 6.

5.7 Extend to Multi-Dimensional Scenarios

In this subsection, we extend PrivCS to multi-dim scenarios. Since tree and grid-based methods are effective in 1-dim and multi-dim settings respectively, we leverage them both to answer multi-dim queries. Specifically, we estimate 1-dim CS trees for all attributes and construct 2-dim adaptive grids. The workflow is as follows.

Step I: Estimating m 1-dim overall CS trees. Each party assigns half of its users to estimate the 1-dim distributions. Specifically, each party P_i randomly allocates $|U^i|/2m$ users to estimate the 1-dim distribution for each attribute and sends the noisy histogram to the server. The server then aggregates the received histograms and constructs the overall CS trees using the same procedure designed for the 1-dim setting across all attributes.

Step II: Estimating $\binom{m}{2}$ 2-dim adaptive grids. We estimate and construct the 2-dim grids for each pair of attributes. Concretely, for each attribute pair (A_ρ, A_ν), where $1 \le \rho \le m$, $1 \le \nu \le m$, and $\rho \ne \nu$, each party P_i randomly selects $|U^i|/2\binom{m}{2}$ of its remaining users to estimate the frequencies of the grid cells via OUE with a privacy budget ϵ. Based on the partition of each 1-dim overall CS tree that is estimated in Step 1, the non-uniform cells are adaptively determined, using the same method with [26]. Each party then uploads the estimated grids to the server for aggregation.

Step III: Collaborative frequency filtering aggregation on grids. Upon receiving all the grids from different parties, the server applies the CF and post-processing to aggregate the grids. In contrast to that in 1-dim settings, we construct the 2-dim grids of each attribute pair based on their 1-dim partitions rather than by directly estimating 2-dim distributions. This is because SW is designed for 1-dim settings. Consequently, we need to modify Equations (6)–(8) to adapt to 2-dim settings. Specifically, β in Equation (6) is computed using the local frequency within a party and the overall frequency across all parties for the same interval, estimated by SW for 1-dim histogram estimation. However, such estimations cannot be directly obtained in 2-dim settings. Therefore, we approximate the desired joint 2-dim distribution using the corresponding marginal 1-dim distributions. That is, we set $\beta = |\hat{f}^i_\rho \cdot \hat{f}^i_\nu - \hat{f}_\rho \cdot \hat{f}_\nu| / \hat{f}_\rho \cdot \hat{f}_\nu$ and feed it to Equation (6) when computing values for the attribute pair (A_ρ, A_ν). Here, $\hat{f}^i_\rho$ denotes the local frequency of the current grid's interval in the 1-way marginal distribution of A_ρ for party P_i, estimated in Step 1, while $\hat{f}_\rho$ represents the overall marginal frequency of the same interval, also estimated in Step 1. For Equations 7 and 8, we similarly use marginal distributions to approximate local and overall frequencies when selecting the threshold $\xi^* = \xi^*_1 \cdot \xi^*_2$, where $\xi^*_1 \in [0, e^{-\epsilon}/5]$ and $\xi^*_2 \in [\xi^*_1, e^{-\epsilon}/5]$. Thus, only grids whose local frequency importance and intervals' overall frequency fall within this range are pruned. Finally, we optimize the frequencies of the grids via the post-processing method introduced in [26].

Step IV: Answering λ-dim federated range queries. We answer a 2-dim query by a $|\mathcal{X}_\rho| \times |\mathcal{X}_\nu|$ response matrix, which encodes the overall 2-dim distribution and is estimated from 1-dim trees and 2-dim grids. Following the previous studies [26, 27, 34], we also apply the maximum entropy and weighted update algorithms, and estimate the distribution using a 2^λ response matrix derived from the associated 2-dim queries for λ-dim ($\lambda > 2$) queries.

6 EXPERIMENTS

6.1 Experimental Setting

Datasets. We conduct experiments over three real-world datasets (*Amazon*, *Instacart*, and *Tokyo*) and three synthetic datasets following different data distributions (*Gaussian*, *Zipf*, and *Hybrid*), each comprising 5 attributes. A brief summary is shown in Table 3. For the *Amazon* review dataset [10], we select stores, timestamps, comments, prices, and ratings from the Fashion category. The *Instacart* dataset [25] contains anonymized grocery orders from Instacart users. We choose products, numbers of add to cart, departments, order hours, and aisle ids for analysis and distribute the orders across four parties. *Tokyo* Check-in Dataset [33] contains check-in data collected from April 2012 to February 2013. We choose venues, venue categories, UTC times, latitudes, and longitudes as attributes. Following the existing studies [16, 35, 37], we employ Dirichlet distribution on the frequencies ($\beta = 0.5$) to simulate realistic non-IID data distribution across parties. The concentration parameter β controls the degree of the distribution skewness. Due to the small $\beta = 0.5$, some parties' users may not have any values in an attribute's subdomain. The *Gaussian* dataset has three parties, and the *Zipf* dataset contains four parties. We set the frequency distributions of different dimensions and parties with varying parameters. The *Hybrid* dataset includes six parties and four kinds of distributions. Table 4 in Appendix A summarizes the details. We partition each attribute's domain into 1024 groups for 1-dim settings or 256 groups for multi-dim settings, unless whose original domain size is smaller than these specified thresholds.

Baselines and parameter setting. We adapt the state-of-the-art LDP-enabled algorithms for answering range queries to the federated setting as baselines, including AHEAD [7], PrivNUD [27], PriPL-Tree [26] for 1-dim cases, and HDG [34], AHEAD, PrivNUD, PRISM [31], PriPL-Tree for multi-dim cases. Specifically, we run the baselines in each party and answer the queries by summing up the estimated counts of all parties according to their user amounts and then dividing by the overall population. We exclude exisitng work HH, HIO, and DHT, as they have been demonstrated to be inferior to others in 1-dim and multi-dim cases [3, 7, 26, 34]. In our experiments, we set $\alpha = 0.3$ being the user proportion for Phase I.

Metric. We evaluate the utility of the mechanisms using the mean square error (MSE) metric following the previous studies [7, 26, 27]. Let $Q \in \mathbf{Q}$ denote a federated range query Q in the query

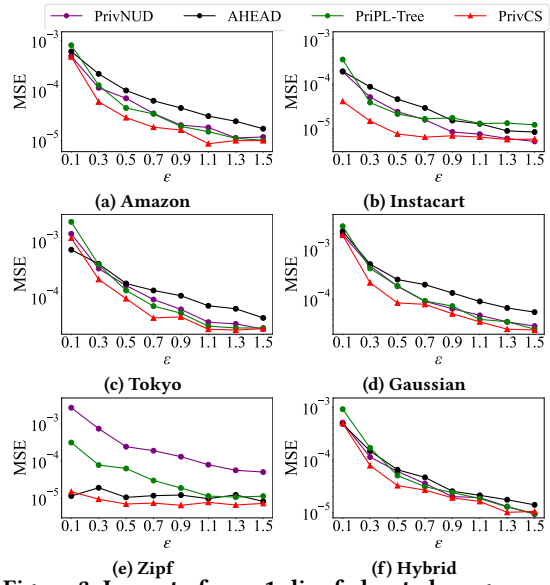


Figure 3: Impact of ϵ on 1-dim federated range queries.

set $\mathcal{Q}$, $\hat{f}_Q$ and f_Q are the estimated frequency and ground truth frequency of Q, the MSE of Q is calculated as follows,

$$MSE(\mathcal{Q}) = \sum_{Q \in \mathcal{Q}} \frac{(\hat{f}_Q - f_Q)^2}{|\mathcal{Q}|}.$$

We randomly generate 1,000 queries, each of which Q has a random query volume $vol(Q)$, which represents the ratio of the query range size to the domain size on each attribute. We calculate the MSE of these queries, where smaller scores indicate better performance. All the mechanisms are executed 50 times and the average is plotted.

6.2 Evaluation Results on 1-dim Query

In this subsection, we evaluate the performance of the proposed PrivCS mechanism based on MSE for 1-dim federated range queries.

Overall results on utility. We evaluate our PrivCS with baselines over six datasets. We set ϵ from 0.1 to 1.5 following the previous studies [7, 27]. As shown in Figure 3, AHEAD performs the worst on most datasets since it not only uses a fixed granularity for decomposition, but also relies on a uniform assumption. PrivNUD applies the adaptive decomposition, but the uniform assumption, especially under the skew distribution of federated settings, leads to poor performance. Although PriPL-Tree is under the linear assumption, it faces similar issues as well. In contrast, PrivCS outperforms baselines across both real-world and synthetic datasets, demonstrating the effectiveness of the private CS fitting and CF aggregation.

Impact study on user allocation ratio. We investigate the performance of the PrivCS mechanism under different user allocation ratios α for Phase I. As shown in Figure 4 (a), we vary α from 0.1 to 0.9 and run PrivCS on *Zipf*, *Gaussian*, and *Hybrid* datasets. It can be observed that the performance is stable under different α, and the optimal performance occurs when $\alpha = 0.3$ (which is the default) or 0.4, illustrating the robustness of the PrivCS against α.

Impact study on domain size. We evaluate PrivCS with baselines on *Gaussian* dataset of different domain sizes. Figure 4 (b) shows that PrivCS outperforms baselines under varying domain sizes. The private CS fitting and the CF aggregation benefit the performance by providing a better approximation of proportion information of different potent queries within a node and being carefully aggregated across different parties.

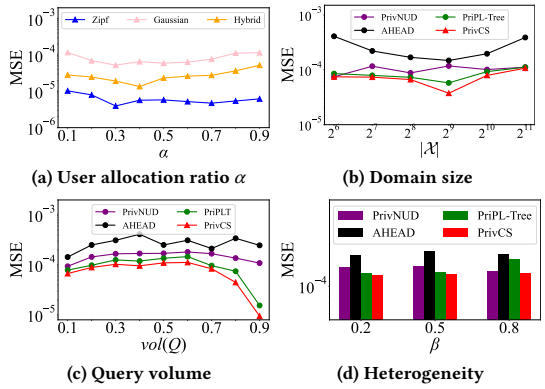


Figure 4: Impact of different parameters on 1-dim federated range queries over Gaussian dataset, with $\epsilon = 0.7$.

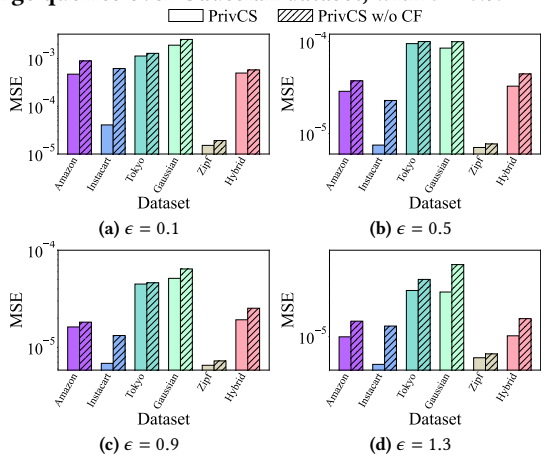


Figure 5: Impact of CF on 1-dim federated range queries.

Impact study on query volume. As shown in Figure 4 (c), we assess the utility of our PrivCS mechanism under varying query volumes $vol(Q)$ from 0.1 to 0.9 on *Gaussian*. The private CS fitting and the CF aggregation enable PrivCS to outperform baselines. The MSE increases when $vol(Q)$ becomes larger and closer to 0.5, as both the ground truth and the estimated frequencies are small, then with increasing $vol(Q)$, the MSE falls quickly since the large volume requires more noisy nodes involved.

Impact study on heterogeneity. We study the impact of the data heterogeneity. We use the Dirichlet distribution with different parameters $\beta \in \{0.2, 0.5, 0.8\}$ to assign the overall distribution to each party with different proportions to simulate varying data skewness. A small β indicates more unbalanced distributions among parties. In Figure 4 (d), we observe that PrivCS consistently outperforms baselines under different distribution skewnesses. The superiority is mainly because the private CS fitting captures the distribution information efficiently and the CF aggregation leads to less noisy estimated frequencies after aggregation, which takes the distribution imbalance across parties into account.

Impact study on the collaborative frequency filtering aggregation. We investigate the effectiveness of the CF aggregation over different datasets, and the results are shown in Figure 5. PrivCS consistently achieves lower MSE compared to scenarios without the CF aggregation (i.e., excluding the calibration and pruning) across all the datasets. It can be observed that the gap is relatively large under *Hybrid*, *Amazon*, and *Instacart*, which involve more users in the experiments. This is because the filtering threshold and the frequencies to be pruned are estimated more accurately, given that

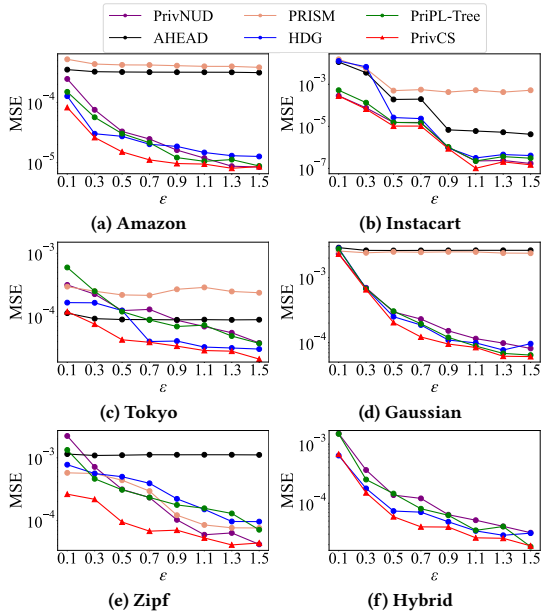


Figure 6: Impact of ϵ on multi-dim federated range queries.

the variance of the frequency estimation protocol is conversely proportional to the involved user population.

6.3 Evaluation Results on Multi-dim Query

In this subsection, we evaluate the performance of our PrivCS mechanism for multi-dim queries.

Overall results on utility. We compare PrivCS with HDG, AHEAD, PrivNUD, PRISM, and PriPL-Tree on real-world and synthetic 5-dim datasets. We randomly generate 1,000 2-dim queries. As shown in Figure 6, PrivCS outperforms baselines under different datasets. AHEAD and PRISM suffer from the non-adaptive decomposition, which introduces even more errors in federated settings. Compared to HDG, PrivNUD, and PriPL-Tree, PrivCS benefits from the private CS fitting and the CF aggregation that provide more reasonable approximation and reduce the negative impact of noisy frequencies as much as possible, thus achieving better performance.

Impact study on data dimension. We study the impact of data dimension. We run the algorithms on *Gaussian* datasets with different data dimensions $m \in \{5, 10, 15, 20, 25, 30\}$ for answering 2-dim queries. The results shown in Figure 7 (a) demonstrate the robustness of PrivCS, which outperforms all baselines under varying data dimensions. The error grows as m increases since there are fewer users to estimate each node's frequency due to the larger domain.

Impact study on query dimension. We randomly generate 1000 λ-dim queries to evaluate the performance of PrivCS under different query dimensions, where $\lambda \in \{2, 3, 4, 5\}$. Figure 7 (b) demonstrates that PrivCS outperforms other algorithms. Notably, the MSEs of all methods decrease as the query dimension increases. This is attributed to the fact that, with higher query dimensions, the number of records within the specified query range is significantly reduced, resulting in a lower frequency.

Impact study on attribute correlation. To better understand the impact of the attribute correlation, we conduct experiments on *Gaussian* datasets with covariances spanning $[0, 1]$. As shown in Figure 7 (c), PrivCS consistently outperforms baseline methods across all correlation levels, demonstrating the superiority and robustness of the private CS fitting and the CF aggregation.

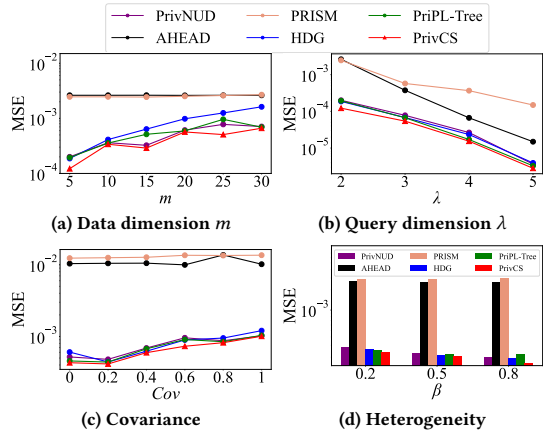


Figure 7: Impact of different parameters on multi-dim federated range queries over Gaussian dataset, with $\epsilon = 0.7$.

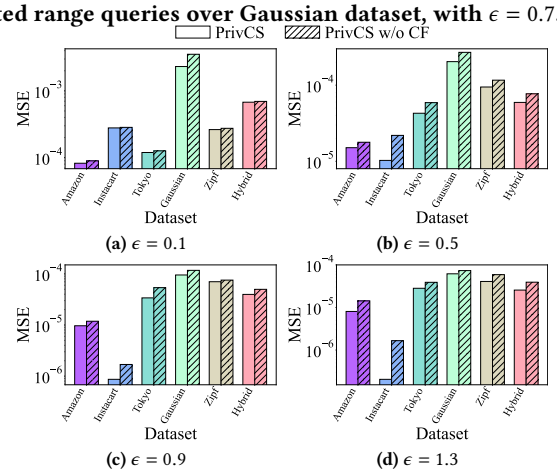


Figure 8: Impact of CF on multi-dim federated range queries.

Impact study on heterogeneity. We investigate the influence of data heterogeneity in multi-dim settings as well. Consistent with our 1-dim experiments, we use the Dirichlet distribution with parameters $\beta \in \{0.2, 0.5, 0.8\}$ to generate varying data skewnesses. Figure 7 (d) shows that PrivCS is also robust under multi-dim settings. This is because the CS fitting and CF aggregation effectively reduce the approximation errors and accumulated LDP noises.

Impact study on the collaborative frequency filtering aggregation. At last, we evaluate if the CF aggregation is effective in answering multi-dim queries. The observation in Figure 8 is similar to that in 1-dim settings. The PrivCS always outperforms the scenario without CF aggregation, demonstrating its effectiveness under more complex multi-dim scenarios.

7 CONCLUSION

In this paper, we study the problem of answering federated range queries over arbitrary data distributions under LDP. We propose a novel private cubic spline (PrivCS) mechanism, in which a private CS fitting is designed to model the underlying data distribution using cubic splines instead of relying on a uniform or a linear assumption. We then design a novel collaborative frequency filtering aggregation that collaboratively calibrates and masks some estimated frequencies to reduce the accumulated noise errors. To the best of our knowledge, this is the first solution to answering federated range queries in a cross-party setting under the stringent ϵ-LDP. Extensive experiments on both synthetic and real-world datasets demonstrate the effectiveness of our mechanism.

REFERENCES

[1] Edwin E. Catmull and Raphael Rom. 1974. A Class of Local Interpolating Splines. In *Computer Aided Geometric Design*. 317–326.

[2] Graham Cormode, Tejas Kulkarni, and Divesh Srivastava. 2018. Marginal Release under Local Differential Privacy. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*. 131–146.

[3] Graham Cormode, Tejas Kulkarni, and Divesh Srivastava. 2019. Answering Range Queries under Local Differential Privacy. In *Proceedings of 45th International Conference on Very Large Data Bases (PVLDB)*, Vol. 12. 1126–1138.

[4] Graham Cormode, Samuel Maddock, and Carsten Maple. 2021. Frequency Estimation under Local Differential Privacy. In *Proceedings of 47th International Conference on Very Large Data Bases (PVLDB)*, Vol. 14. 2046–2058.

[5] Graham Cormode, Cecilia Procopiuc, Divesh Srivastava, Entong Shen, and Ting Yu. 2012. Differentially Private Spatial Decompositions. In *Proceedings of the IEEE 28th International Conference on Data Engineering (ICDE)*. 20–31.

[6] Wolfgang Dahmen, Bin Han, R-Q Jia, and Angela Kunoth. 2000. Biorthogonal Multiwavelets on the Interval: Cubic Hermite Splines. In *Constructive Approximation*, Vol. 16. 221–259.

[7] Linkang Du, Zhikun Zhang, Shaojie Bai, Changchang Liu, Shouling Ji, Peng Cheng, and Jiming Chen. 2021. AHEAD: Adaptive Hierarchical Decomposition for Range Query under Local Differential Privacy. In *Proceedings of the 28th ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 1266–1288.

[8] Cynthia Dwork, Aaron Roth, et al. 2014. The Algorithmic Foundations of Differential Privacy. In *Foundations and Trends® in Theoretical Computer Science (TCS)*, Vol. 9. 211–407.

[9] Michael Hay, Vibhor Rastogi, Gerome Miklau, and Dan Suciu. 2010. Boosting the Accuracy of Differentially Private Histograms through Consistency. In *Proceedings of 36th International Conference on Very Large Data Bases (PVLDB)*, Vol. 3. 1021–1032.

[10] Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiusi Chen, and Julian McAuley. 2024. Bridging Language and Items for Retrieval and Recommendation.

[11] Ziyue Huang and Ke Yi. 2021. Approximate Range Counting under Differential Privacy. In *37th International Symposium on Computational Geometry (SoCG)*. 45:1–45:14.

[12] Shiva Prasad Kasiviswanathan, Homin K Lee, Kobbi Nissim, Sofya Raskhodnikova, and Adam Smith. 2011. What Can We Learn Privately?. In *SIAM Journal on Computing (SICOMP)*, Vol. 40. 793–826.

[13] Gary D Knott. 1999. *Interpolating Cubic Splines*. Vol. 18. Springer Science & Business Media.

[14] Anastasios M. Lekkas and Thor I. Fossen. 2014. Integral LOS Path Following for Curved Paths Based on a Monotone Cubic Hermite Spline Parametrization. In *IEEE Transactions on Control Systems Technology*, Vol. 22. 2287–2301.

[15] Chao Li, Michael Hay, Gerome Miklau, and Yue Wang. 2014. A Data- and Workload-Aware Algorithm for Range Queries under Differential Privacy. In *Proceedings of 40th International Conference on Very Large Data Bases (PVLDB)*, Vol. 7. 341–352.

[16] Qinbin Li, Yiqun Diao, Quan Chen, and Bingsheng He. 2022. Federated Learning on Non-IID Data Silos: An Experimental Study. In *Proceedings of the IEEE 38th International Conference on Data Engineering (ICDE)*. 965–978.

[17] Xiaochen Li, Weiran Liu, Jian Lou, Yuan Hong, Lei Zhang, Zhan Qin, and Kui Ren. 2024. Local Differentially Private Heavy Hitter Detection in Data Streams with Bounded Memory. In *Proceedings of the 2024 ACM Conference on Management of Data (SIGMOD)*. 1–27.

[18] Zitao Li, Tianhao Wang, Milan Lopuhaä-Zwakenberg, Ninghui Li, and Boris Škoric. 2020. Estimating Numerical Distributions under Local Differential Privacy. In *Proceedings of the 2020 ACM Conference on Management of Data (SIGMOD)*. 621–635.

[19] Ryan McClarren. 2017. *Computational Nuclear Engineering and Radiological Science using Python*. Academic Press.

[20] Ryan McKenna, Raj Kumar Maity, Arya Mazumdar, and Gerome Miklau. 2020. A Workload-Adaptive Mechanism for Linear Queries under Local Differential Privacy. In *Proceedings of 46th International Conference on Very Large Data Bases (PVLDB)*, Vol. 13. 1905–1918.

[21] Wahbeh Qardaji, Weining Yang, and Ninghui Li. 2013. Differentially Private Grids for Geospatial Data. In *Proceedings of the IEEE 29th International Conference on Data Engineering (ICDE)*. 757–768.

[22] Wahbeh Qardaji, Weining Yang, and Ninghui Li. 2014. PriView: Practical Differentially Private Release of Marginal Contingency Tables. In *Proceedings of the 2014 ACM Conference on Management of Data (SIGMOD)*. 1435–1446.

[23] Xuebin Ren, Chia-Mu Yu, Weiren Yu, Shusen Yang, Xinyu Yang, Julie A. McCann, and Philip S. Yu. 2018. LoPub : High-Dimensional Crowdsourced Data Publication with Local Differential Privacy. In *IEEE Transactions on Information Forensics and Security (TIFS)*, Vol. 13. 2151–2166.

[24] Jiaqi Shao, Shanshan Han, Chaoyang He, and Bing Luo. 2023. Privacy-Preserving Federated Heavy Hitter Analytics for Non-IID Data. In *ICML Workshop on Federated Learning and Analytics in Practice (FL-ICML)*.

[25] Jeremy Stanley, Meg Risdal, Sharathrao, and Will Cukierski. 2017. Instacart Market Basket Analysis. https://kaggle.com/competitions/instacart-market-basket-analysis. Kaggle.

[26] Leixia Wang, Qingqing Ye, Haibo Hu, and Xiaofeng Meng. 2024. PriPL-Tree: Accurate Range Query for Arbitrary Distribution under Local Differential Privacy. In *Proceedings of 50th International Conference on Very Large Data Bases (PVLDB)*, Vol. 17. 3031–3044.

[27] Ning Wang, Yaohua Wang, Zhigang Wang, Jie Nie, Zhiqiang Wei, Peng Tang, Yu Gu, and Ge Yu. 2023. PrivNUD: Effective Range Query Processing under Local Differential Privacy. In *IEEE 39th International Conference on Data Engineering (ICDE)*. 2660–2672.

[28] Tianhao Wang, Jeremiah Blocki, Ninghui Li, and Somesh Jha. 2017. Locally Differentially Private Protocols for Frequency Estimation. In *Proceedings of the 26th USENIX Security Symposium (USENIX Security)*. 729–745.

[29] Tianhao Wang, Bolin Ding, Jingren Zhou, Cheng Hong, Zhicong Huang, Ninghui Li, and Somesh Jha. 2019. Answering Multi-Dimensional Analytical Queries under Local Differential Privacy. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*. 159–176.

[30] Tianhao Wang, Ninghui Li, and Somesh Jha. 2019. Locally Differentially Private Heavy Hitter Identification. In *IEEE Transactions on Dependable and Secure Computing (TDSC)*, Vol. 18. 982–993.

[31] Yufei Wang and Xiang Cheng. 2022. PRISM: Prefix-Sum based Range Queries Processing Method under Local Differential Privacy. In *IEEE 38th International Conference on Data Engineering (ICDE)*. 433–445.

[32] Xiaokui Xiao, Guozhang Wang, and Johannes Gehrke. 2010. Differential Privacy via Wavelet Transforms. In *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, Vol. 23. 1200–1214.

[33] Dingqi Yang, Daqing Zhang, Vincent W. Zheng, and Zhiyong Yu. 2014. Modeling User Activity Preference by Leveraging User Spatial Temporal Characteristics in LBSNs. *IEEE Transactions on Systems, Man, and Cybernetics: Systems (TSMC)* 45, 1, 129–142.

[34] Jianyu Yang, Tianhao Wang, Ninghui Li, Xiang Cheng, and Sen Su. 2020. Answering Multi-Dimensional Range Queries under Local Differential Privacy. In *Proceedings of 46th International Conference on Very Large Data Bases (PVLDB)*. 378–390.

[35] Mikhail Yurochkin, Mayank Agarwal, Soumya Ghosh, Kristjan Greenewald, Nghia Hoang, and Yasaman Khazaeni. 2019. Bayesian Nonparametric Federated Learning of Neural Networks. In *International Conference on Machine Learning (ICML)*. 7252–7261.

[36] Jun Zhang, Xiaokui Xiao, and Xing Xie. 2016. PrivTree: A Differentially Private Algorithm for Hierarchical Decompositions. In *Proceedings of the 2016 ACM Conference on Management of Data (SIGMOD)*. 155–170.

[37] Yuemin Zhang, Qingqing Ye, and Haibo Hu. 2025. Federated Heavy Hitter Analytics with Local Differential Privacy. In *Proceedings of the 2025 ACM Conference on Management of Data (SIGMOD)*, Vol. 3. Article 42, 27 pages.

[38] Zhikun Zhang, Tianhao Wang, Ninghui Li, Shibo He, and Jiming Chen. 2018. CALM: Consistent Adaptive Local Marginal for Marginal Release under Local Differential Privacy. In *Proceedings of the 25th ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 212–229.

A APPENDIX

Table 4: Synthetic dataset details.

Dataset	Dimension 1	Dimension 2	Dimension 3	Dimension 4	Dimension 5	# Users	# Total Users
Gaussian	Gaussian(0, 2)	Gaussian(0, 5)	Gaussian(0, 2)	Gaussian(0, 5)	Gaussian(0, 10)	50,000	300,000
	Gaussian(0, 1)	Gaussian(0, 7)	Gaussian(0, 3)	Gaussian(0, 10)	Gaussian(0, 2)	150,000	
	Gaussian(0, 1.5)	Gaussian(0, 5)	Gaussian(0, 2)	Gaussian(0, 2)	Gaussian(0, 10)	150,000	
Zipf	Zipf(1.5)	Zipf(1.3)	Zipf(1.1)	Zipf(1.01)	Zipf(1.5)	9,746	80,677
	Zipf(1.3)	Zipf(1.5)	Zipf(1.3)	Zipf(1.1)	Zipf(1.01)	17,900	
	Zipf(1.1)	Zipf(1.01)	Zipf(1.5)	Zipf(1.3)	Zipf(1.1)	23,824	
	Zipf(1.01)	Zipf(1.1)	Zipf(1.01)	Zipf(1.5)	Zipf(1.3)	29,207	
Hybrid	Cauchy(1.0)	Cauchy(1.0)	Cauchy(1.0)	Cauchy(1.0)	Cauchy(1.0)	30,000	909,000
	Zipf(1.01)	Zipf(1.01)	Zipf(1.01)	Zipf(1.01)	Zipf(1.01)	29,000	
	Gamma(1.5, 1000)	Gamma(1.5, 1000)	Gamma(1.5, 1000)	Gamma(1.5, 1000)	Gamma(1.5, 1000)	100,000	
	Gaussian(0, 1.5)	Gaussian(0, 1.5)	Gaussian(0, 1.5)	Gaussian(0, 1.5)	Gaussian(0, 1.5)	100,000	
	Gaussian(0, 1)	Gaussian(0, 1)	Gaussian(0, 1)	Gaussian(0, 1)	Gaussian(0, 1)	150,000	
	Gamma(10, 1000)	Gamma(10, 1000)	Gamma(10, 1000)	Gamma(10, 1000)	Gamma(10, 1000)	500,000	

A.1 Post-Processing Steps

In the weighted averaging step, we traverse nodes in the overall CS tree from the leaves to the root, minimizing the frequency variance for each node. For a leaf node n_k, which has frequencies $\hat{f}_k$ (from node frequency estimation) and $\tilde{f}_k$ (from the noisy histogram estimation), we update its frequency to $\hat{f}_k = \theta\hat{f}_k + (1-\theta)\tilde{f}_k$, where $\theta = \frac{\mathrm{Var}(\tilde{f}_k)}{\mathrm{Var}(\hat{f}_k)+\mathrm{Var}(\tilde{f}_k)}$, to result in the minimal variance of

$Var(\hat{f}_k) = \frac{Var(\hat{f}_k)Var(\tilde{f}_k)}{Var(\hat{f}_k)+Var(\tilde{f}_k)}$. For each non-leaf node n_k, we similarly refine its frequency using its estimated frequency $\hat{f}_k$ and the sum of its child nodes' frequencies $\sum_{n_{cd} \in \text{child}(n_k)} \hat{f}_{cd}$. We then apply the frequency consistency step that updates frequencies from the root to leaves. Note that the frequency of the root does not need to be estimated and optimized, since it represents the entire domain so that the frequency is always 1. Given a parent node n_p with optimized frequency $\hat{f}_p \geq 0$ and its child nodes' frequencies $\{\hat{f}'_{cd} | n_{cd} \in \text{child}(p)\}$ that to be optimized, we solve the following equation: $\min \sum_{n_{cd} \in \text{child}(n_p)} \left(\hat{f}_{cd} - \hat{f}'_{cd}\right)^2$, subject to $\sum_{n_{cd} \in \text{child}(n_p)} \hat{f}_{cd} = \hat{f}_p$ and $\forall n_{cd} \in \text{child}(n_p), \hat{f}_{cd} \geq 0$. Let D^+ denote the set of child nodes with positive updated frequencies, and D_0 be those with zero updated frequencies, then the optimal frequency is $\hat{f}_{cd} = \hat{f}_{cd} + \left(\hat{f}_p - \sum_{n_j \in D^+} \hat{f}_j\right) / |D^+|$ if $n_{cd} \in D^+$, and $\hat{f}_{cd} = 0$ if $n_{cd} \in D_0$.

A.2 Privacy Proof

We prove that the proposed PrivCS mechanism satisfies ϵ-LDP.

Theorem A.1. *The PrivCS mechanism satisfies ϵ-LDP.*

Proof. The PrivCS mechanism consists of three phases, private CS fitting, PrivCS tree construction, and collaborative frequency filtering aggregation.

In the first phase, each party requires a subset of its users to estimate the noisy histogram. These users perturb their data locally via SW with the privacy budget ϵ and upload the perturbed data to their respective parties. The parties then send the estimated histograms to the server. Since SW satisfies ϵ-LDP, the estimated results on both the party and server sides satisfy LDP. Based on the post-processing theorem, fixing the breakpoints and partitions by minimizing the interpolating errors also satisfies ϵ-LDP, as all the parameters used are either derived from noisy results or chosen independently.

In the construction phase, let $N-1$ denote the number of non-root nodes in the CS tree. Each party randomly divides its remaining users into $N-1$ groups, which do not raise privacy concerns. Each user in a different group estimates the frequencies of the corresponding node (representing a different interval) by perturbing their value via OUE with privacy budget ϵ. They then upload the perturbed values to their respective parties, which send the aggregated estimations to the server. This phase satisfies LDP, as OUE itself satisfies LDP and all parameters used in the reduction and user allocation are privacy-free.

In the aggregation phase, we apply the collaborative frequency filtering, weighted averaging, and the frequency consistency step based on independently determined parameters and the noisy results estimated by SW and OUE. Therefore, the aggregation phase also satisfies LDP according to the post-processing theorem. Thus, the proof is complete. □